

Subodh Neupane
1001995258

3: Consider the function $f(x,y) = \sin(\cos(x)+\sin(2y))$.

- Part a (4 points): what is the partial derivative of f with respect to x ? Express your solution as a function of x and y , and show how you derive it.

→ $f(x,y) = \sin(\cos(x)+\sin(2y))$
→ Let $\cos(x)+\sin(2y)$ be u
→ Now, $d/dy(\sin(u)) = \cos(u)$
→ Replace u with the value and apply chain rule,
→ $\cos(\cos(x)+\sin(2y)) \cdot d/dx(\cos(x)+\sin(2y))$
→ Differentiate with respect to d/dx , $\cos(\cos(x)+\sin(2y)) \cdot -\sin x$

→ $-\cos(\cos x + \sin(2y)) \sin x$

- Part b (4 points): what is the partial derivative of f with respect to y ? Express your solution as a function of x and y , and show how you derive it.

$$f(x,y) = \sin(\cos(x)+\sin(2y))$$

→ Let $\cos(x)+\sin(2y)$ be u
→ Now, $d/dy(\sin(u)) = \cos(u)$
→ Replace u with the value and apply chain rule,
→ $\cos(\cos(x)+\sin(2y)) \cdot d/dy(\cos(x)+\sin(2y))$
→ Differentiate with respect to d/dy , $\cos(\cos(x)+\sin(2y)) \cdot (0+2\cos(2y))$

→ $\cos(\cos x + \sin(2y)) \cdot 2\cos(2y)$

- Part c (2 points): What is the gradient of foo? Express the gradient as a function of x and y, that outputs a 2-dimensional vector.

Gradient of foo = (dfoo/dx,dffo/dy)

$$\rightarrow (-\cos(\cos x + \sin(2y))\sin x, \cos(\cos x + \sin(2y)) * 2\cos(2y))$$

The chain rule can be useful in computing partial derivatives here. The slides on [training perceptrons](#) and [training neural networks](#) show many examples of using the chain rule.

5: Note: In this question you should assume that the activation function of a perceptron is the step function. More specifically, this function:

- outputs 0 if the weighted sum of inputs is LESS THAN 0 (not less than or equal).
- outputs 1 if the weighted sum of inputs is greater than or equal to 0.

Design a perceptron that takes three Boolean inputs (i.e., inputs that are equal to 0 for false, and 1 for true), and outputs: 1 if at least two of the three inputs are true, 0 otherwise. You should NOT worry about what your perceptron does when the input values are not 0 or 1.

Your answer can be a drawing, as in the slides, or it can simply be text. Either way, your answer should clearly specify the values for the bias weight and the three regular weights.

Ans:

w1 (first input weight): 1

w2 (second input weight): 1

w3 (third input weight): 1

Bias weight: -1.5

The perceptron will output 1 when the weighted sum is ≥ 0 , which occurs when at least two inputs are 1.

Truth table verification:

- (0,0,0) = 0 (weighted sum = -1.5)
- (0,0,1) = 0 (weighted sum = -0.5)
- (0,1,0) = 0 (weighted sum = -0.5)
- (0,1,1) = 1 (weighted sum = 0.5)
- (1,0,0) = 0 (weighted sum = -0.5)
- (1,0,1) = 1 (weighted sum = 0.5)
- (1,1,0) = 1 (weighted sum = 0.5)
- (1,1,1) = 1 (weighted sum = 1.5)

6: Note: In this question you should assume that the activation function of a perceptron is the step function, same as in Task 5.

Design a neural network that:

- takes two inputs, A and B, that can be any real numbers.
- outputs 1 if $2A + 3B = 4$.
- outputs 0 otherwise.

Your answer can be a drawing, as in the slides, or it can simply be text. Either way, your answer should clearly specify:

- How the network is structured, what units it contains, how those units are organized in layers.
- All edges connecting outputs of one layer to inputs in the next layer.
- The values of all weights (including bias weights) of all perceptrons. It should be clear which weight value corresponds to which edge in the network.

Ans:

Inputs: A, B

Weights:

- w_1 (weight for input A) = 2
- w_2 (weight for input B) = 3
- bias weight (b) = -4

Weighted Sum: The perceptron calculates the weighted sum: $\text{weighted_sum} = (2 * A) + (3 * B) - 4$

Activation function:

Output = 1 if $\text{weight_sum} \geq 0$

Output = 0 if $\text{weight_sum} < 0$

Test Case:

A = 0, B = 0

- $2(0) + 3(0) - 4 = -4$
- Output: 0 (since $-4 < 0$)

$A = 2, B = 0$

- $2(2) + 3(0) - 4 = 0$
- Output: 1 (since $0 \geq 0$)

$A = 0, B = 4/3$

- $2(0) + 3(4/3) - 4 = 0$
- Output: 1 (since $0 \geq 0$)

$A = 1, B = 2/3$

- $2(1) + 3(2/3) - 4 = 0$
- Output: 1 (since $0 \geq 0$)

$A = 0, B = 1$

- $2(0) + 3(1) - 4 = -1$
- Output: 0 (since $-1 < 0$)

7: Note: In this question you should assume that the activation function of a perceptron is the step function, same as for Tasks 5 and 6.

Is it possible to design a neural network (which could be a single perceptron or a larger network), that satisfies these specs?

- Takes a single input called X , which can be any real number.
- If $X < 3$, the network outputs 0.
- If $3 < X < 7$, the network outputs 1.
- If $X > 7$, the network outputs 0.

We don't care what the network outputs when $X = 3$ or $X = 7$.

If your answer is no, explain why not. If your answer is yes, your solution should fully specify, either using a drawing or in text, the network, as described in Task 6, by specifying the units, layers, connections between layers, and values of weights (including bias weights).

Ans :

Yes, it is possible to design a neural network that satisfies the given specifications using a multi-layer perceptron (MLP) with step function activations.

1. Input Layer: One node representing X .
2. Hidden Layer: Two perceptrons (P1 and P2).
3. Output Layer: One perceptron (P3).

Weights and Connections:

- Input to Hidden:
 - P1: Weight for $X = 1$, Bias = -3.00001 (or -3 , with a strictly-less-than step function). This perceptron activates when $X > 3$.
 - P2: Weight for $X = -1$, Bias = 7.00001 (or 7 , with a strictly-less-than step function applied to $-X$). This perceptron activates when $X < 7$.

- Hidden to Output:
 - P3: Weight for P1 = 1, Weight for P2 = 1, Bias = -1. This perceptron activates only when *both* P1 and P2 are active (output 1).
1. $X < 3$ (e.g., $X = 0, 1, 2$):
 - P1: 0 (because $X < 3$)
 - P2: 0 (because $X < 7$)
 - P3: 0 (because P1 is 0)
 2. $3 \leq X < 7$ (e.g., $X = 3, 4, 5, 6$):
 - P1: 1 (because $X \geq 3$)
 - P2: 0 (because $X < 7$)
 - P3: 1 (because P1 is 1 and P2 is 0)
 3. $X \geq 7$ (e.g., $X = 7, 8, 9$):
 - P1: 1 (because $X \geq 3$)
 - P2: 1 (because $X \geq 7$)
 - P3: 0 (because P2 is 1, not 0)

The hidden layer perceptrons, P1 and P2, create the two necessary boundaries. The output perceptron, P3, acts as a sort of "AND" gate, activating only when *both* P1 and P2 are active, effectively isolating the region between the two boundaries.